

1. Write a Node class with an attribute to store the data of a generic type and another attribute acting as the pointer to the next Node. Let the Node class have a constructor with a parameter of generic type. The Node class should implement the Comparable class.
2. Write a class named MyLinkedList with three attributes, head, tail, and size. MyLinkedList class has Node in Question 1 as its attribute.

| Method Name | Specification                                                                                                                                                                                                                            |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Constructor | An empty default constructor.                                                                                                                                                                                                            |
| getSize     | Return the size of the linkedlist                                                                                                                                                                                                        |
| addFirst    | Add a node to the first position of the linkedlist and return nothing                                                                                                                                                                    |
| addLast     | Add a node to the last position of the linkedlist and return nothing                                                                                                                                                                     |
| removeFirst | Remove the first node in the linkedlist and return the element stored in the node                                                                                                                                                        |
| toString    | Print out the size of the list, followed by the element in each node. If the linkedlist has three nodes with strings "Hi", "How're", and "you?", then calling the toString method will print out:<br>[size=3] >> Hello >> How're >> you? |
| contains    | Return true if the element is within the list                                                                                                                                                                                            |
| clear       | Empty the list and display a statement of "The list is cleared."                                                                                                                                                                         |
| combine     | Combine the elements of two linkedlist in ascending order and return a new linkedlist.                                                                                                                                                   |

3. Declares three MyLinkedList to store Integers, namely list1, list2, and list3. The list1 is a list with 5 numbers in incremental order: 2, 10, 38, 41, 51, whereas list2 with 7 numbers in the same incremental order: 5, 12, 18, 21, 35, 41, 56. The list3 will be the combination of list1 and list2.

Print out list1, list2, and newList.

Here is an example output:

```

public class Tester {

    public static void main(String[] args) {
        MyLinkedList<Integer> list1 = new MyLinkedList<>();
        list1.addLast(2);
        list1.addLast(10);
        list1.addLast(38);
        list1.addLast(41);
        list1.addLast(51);
        MyLinkedList<Integer> list2 = new MyLinkedList<>();
        list2.addLast(5);
        list2.addLast(12);
        list2.addLast(18);
        list2.addLast(21);
        list2.addLast(35);
        list2.addLast(41);
        list2.addLast(56);
        MyLinkedList<Integer> list3 = list1.combine(list1, list2);
        System.out.println(list1.toString());
        System.out.println(list2.toString());
        System.out.println(list3.toString());
    }
}

```

```

[size=5] >> 2 >> 10 >> 38 >> 41 >> 51
[size=7] >> 5 >> 12 >> 18 >> 21 >> 35 >> 41 >> 56
[size=12] >> 2 >> 5 >> 10 >> 12 >> 18 >> 21 >> 35 >> 38 >> 41 >> 41 >> 51 >> 56
BUILD SUCCESSFUL (total time: 1 second)

```

\*\*\*\*\*

4. Create a class named DoublyLinkedList.

| Method Name | Specification                                                                            |
|-------------|------------------------------------------------------------------------------------------|
| Constructor | An empty default constructor.                                                            |
| getSize     | Return the size of the doubly linkedlist                                                 |
| addFirst    | Add a node to the first position of the doubly linkedlist and return nothing             |
| addLast     | Add a node to the last position of the doubly linkedlist and return nothing              |
| removeFirst | Remove the first node in the doubly linkedlist and return the element stored in the node |

|          |                                                                                                                                                                                                                                                                          |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| toString | Print out the size of the list, followed by the element in each node. If the doubly linkedlist has three nodes with strings "Hi", "How're", and "you?", calling the toString method will print out:<br>[size=3] >> Hello >> How're >> you?<br><< you? << How're << Hello |
| contains | Return true if the element is within the list                                                                                                                                                                                                                            |
| clear    | Empty the list and display a statement of "The list is cleared."                                                                                                                                                                                                         |

Here's the sample run:

```
DoublyLinkedList<String> dlist1 = new DoublyLinkedList<>();
dlist1.addLast("wedges");
dlist1.addLast("chips");
dlist1.addLast("french fries");
dlist1.addLast("mashed potatoes");
System.out.println(dlist1.toString());
```

```
[size=4] >> wedges >> chips >> french fries >> mashed potatoes
<< mashed potatoes << french fries << chips << wedges
BUILD SUCCESSFUL (total time: 1 second)
```